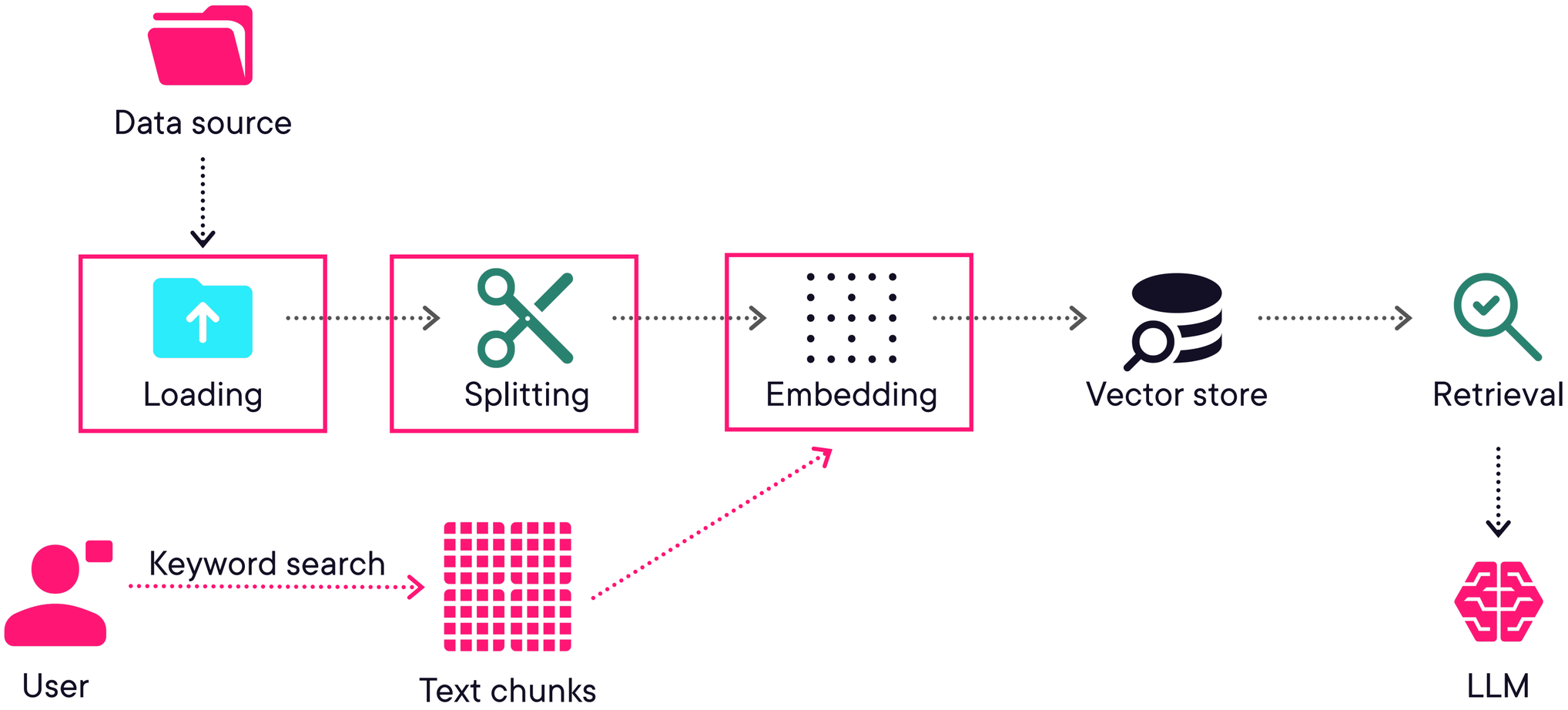
Understanding embeddings and vector representations

Deepali Srivastava
Author

linkedin.com/in/deepali-srivastava/

Why embeddings are needed

Finding the right information for the LLM



Limitations of keyword search

Semantic Understanding

Computers do not understand meaning like humans do

Keyword mismatch

Query: Is he ill?

He is feeling unwell.

He is not in good health.

He is sick.

Need for embeddings

Systems need to understand the meaning of text, not just match exact words

Embeddings as vector representations

Understanding vectors

Vector: a list of numbers
e.g., [0.12, -0.45, 0.78, 0.33, ...]

Each value represents
one dimension

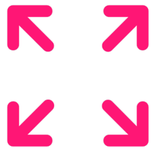
Vectors can be visualized as
points in space

Similarity between vectors can
be measured

Introduction to embeddings

Text is converted into vectors using embedding models

“I am very happy” → [0.26, 0.45, 0.73, 0.38, 0.76, 021, ...]



Each dimension captures some aspect of the text, and together all dimensions represent its overall meaning

---

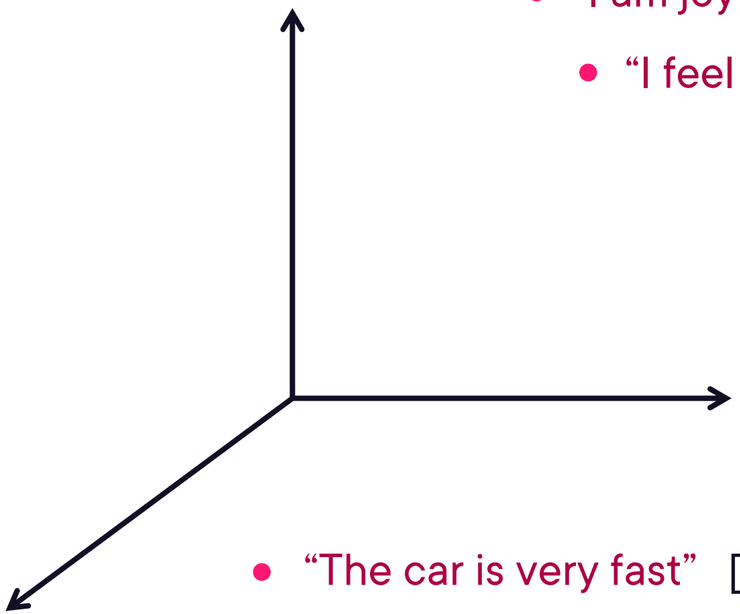
Embeddings are dense vectors and capture semantic meaning

Embeddings in high-dimensional space

Embeddings have fixed size with hundreds of dimensions

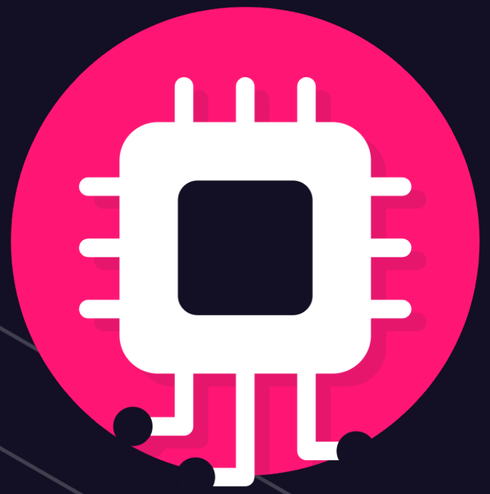
Each text becomes a point in high-dimensional space

Semantic similarity in vector space



- “I am joyful” [0.23, 0.47, 0.80, ...]
  - “I feel very happy” [0.21, 0.45, 0.78, ...]
- “The car is very fast” [0.75, 0.12, 0.20, ...]

Real embeddings are high-dimensional



Embeddings are generated using embedding models

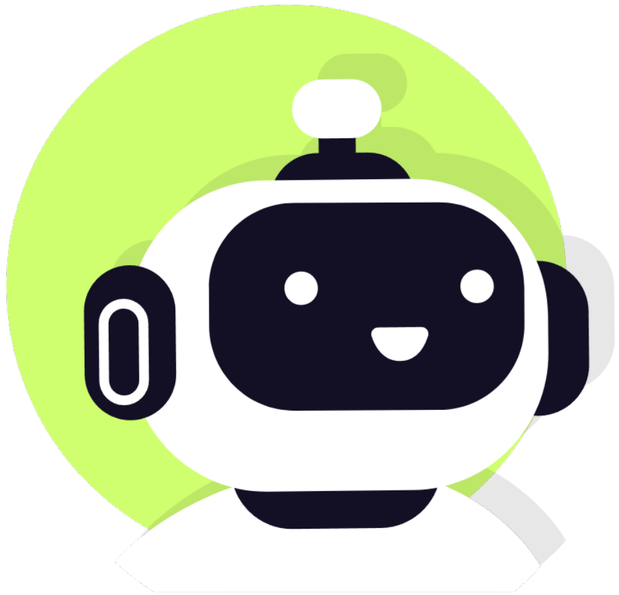
Models learn relationships during training

Similar meaning → similar vectors

Embeddings for different levels of text

Pre-trained models available

- OpenAI
- Hugging Face
- Cohere
- Google



Embeddings used across many AI applications

- Semantic search engines
- Retrieval-augmented generation (RAG)
- Recommendation systems
- Text similarity/duplicate detection
- Clustering

Embedding models in practice with LangChain

Why you need a unified embedding interface

Different APIs

Different providers have different APIs and formats

Need for simplification

Direct integration makes code complex and switching difficult

LangChain embedding abstraction

Same methods across different providers

`embed_query()`
`embed_documents()`

Easy to switch models without changing application logic

Demo

Generating embeddings using OpenAI

Demo

Generating embeddings using Hugging Face

Understanding similarity between embeddings

Comparing embeddings to find relevant information

Query and document embeddings are compared to find relevant results

Similarity measures determine how close or similar vectors are

Common similarity measures: Euclidean distance, cosine similarity, dot product

Common similarity measures for comparing embeddings

Euclidean distance

$d = \text{sqrt}(\sum(A_i - B_i)^2)$

Smaller distance means higher similarity

Cosine similarity

$\cos \theta = (A \cdot B) / (|A||B|)$

A value closer to one indicates higher similarity

Dot product

$A \cdot B = ||A||\ ||B|| \cos(\theta)$

Higher values indicate higher similarity

Using the same embedding model for consistent similarity

Embeddings are generated for both documents and user queries

The same embedding model must be used for both

Different models create different vector spaces, making similarity unreliable

Demo

Computing similarity between query and documents

Embedding dimensions and performance tradeoffs

Understanding embedding dimensions

Embeddings are vectors where each value represents a dimension

Common sizes include 384, 768, and 1536 dimensions

More dimensions allow the model to represent more detailed meaning

Small vs. large embedding models

Small embedding models

384 to 768 dimensions

Example: all-MiniLM-L6-v2

Fast and lower cost

Require less memory and storage

Less detailed semantic information

Large embedding models

1024 to 3072 dimensions

Example: text-embedding-3-large

Slower and more expensive

Require more memory and storage

Better semantic understanding

Tradeoffs in real-world embedding systems

Quality: Larger embeddings capture meaning more accurately

Speed: Smaller embeddings are faster to compute

Cost: Larger models are more expensive to use

Memory: Higher dimensions require more RAM

Storage: More dimensions increase storage requirements

Choosing the right embedding model

Choice depends on system requirements

Smaller models for speed and scalability

Larger models for higher accuracy

Start with a smaller model and move to a larger one if needed

Evaluating embedding quality and limitations

Understanding embedding quality

Good embeddings place similar texts close and different texts far apart

Better embedding quality improves retrieval

Evaluating embeddings

Similarity-based evaluation

Relevant documents should have higher similarity scores than irrelevant ones

Ranking-based evaluation

Checks whether relevant documents appear in the top K results using metrics like Recall@K and Precision@K

Recall@K

How many relevant documents appear in top K results?

$$\text{Recall@K} = \frac{\text{Number of relevant documents in top K}}{\text{Total number of relevant documents}}$$

Example:

- Total of 5 relevant documents in the dataset
- 3 of them are found
- Recall@10 = 3 / 5 = 0.6
- System retrieved 60% in the top 10 results

High recall → most relevant information is retrieved
Low recall → some information is missed

Precision@K

How many of the top K results are relevant?

$$\text{Precision@K} = \frac{\text{Number of relevant documents in top K}}{\text{K}}$$

Example:

- Top 10 results
- 3 documents are relevant
- Precision@10 = 3 / 10 = 0.3
- Only 30% of results are relevant

High precision → most retrieved results are relevant
Low precision → many irrelevant results are included

Recall vs. precision

Recall@K

- Completeness
- Did you find all relevant documents?
- Punishes missing information
- Use when missing results is detrimental (e.g., medical search, legal docs)

Precision@K

- Accuracy
- Are the retrieved documents correct?
- Punishes noise
- Use when wrong results are detrimental (e.g., search UI)

Effect of K on recall and precision

Increasing K improves recall by retrieving more relevant documents

Increasing K may reduce precision due to more irrelevant results

K is usually kept small (3 to 20) depending on the use case

Demo

Ranking-based evaluation

Limitations of embedding

Ambiguity: Same word, different meanings
“I ate an apple,” “Apple released a new iPhone,” “Apple is growing”

Domain-specific meaning: Same term across fields
“Transformer improved the model,” “Transformer reduced voltage”

Negation confusion: Opposite meanings appear similar
“He is happy,” “He is not happy”

Factual correctness: Similar ≠ true
“Who discovered gravity?” “Newton discovered gravity,”
“Einstein discovered gravity”